

Rapport – TP Java Générateur de QR Code (parties 1 et 2)

1. Présentation de l'application

Application Java Swing qui génère un QR code à partir d'une information saisie (texte, lien, e-mail, téléphone) et l'intègre dans un fichier PDF personnalisable.

Élément	Choix
Langage	Java 17
Interface	Java Swing
Architecture	MVC
Génération PDF	iText 5.5.13
Génération QR code	ZXing 3.5.3
Sauvegarde	Gson 2.11 (fichiers JSON)
Tests	JUnit 5
Projet	Maven, ouvert dans IntelliJ IDEA

2. Architecture MVC

Couche	Package	Classes	Rôle
Modèle (données)	model	Projet, Profil, ImagePdf, TypeContenu, AlignementQr, NiveauCorrection, Couleurs	Contient les données et leurs règles de validation
Modèle (traitements)	service	QrCodeGenerator, PdfGenerator, SauvegardeService	Génère le QR code, le PDF, lit et écrit les fichiers
Vue	view	MainView, BoutonCouleur, ImageTableModel	Affiche la fenêtre, ne contient aucun traitement
Contrôleur	controller	MainController	Reçoit les actions de la vue, appelle le modèle, affiche le résultat ou l'erreur
Exceptions	exception	QrAppException et 4 sous-classes	Erreurs de l'application avec un message clair

Main crée la vue, les services et le contrôleur, puis affiche la fenêtre.

3. Fonctionnement

Utilisation

- Onglet **Contenu** : saisir le titre, le type de contenu, le contenu du QR code et la description. L'aperçu du QR code se met à jour pendant la saisie.
- Onglet **Style** : choisir la police, les tailles, le style et les couleurs. Le profil peut être sauvegardé et rechargé.
- Onglet **Images** : ajouter des images et régler leur position et leur taille en millimètres.
- Cliquer sur **Générer le PDF** : choix du fichier, barre de progression, puis proposition d'ouvrir le PDF.
- Menu **Fichier** : enregistrer ou ouvrir un projet.

Fichier Profil Aide

Contenu

Style

Images

Titre du PDF :

Portes ouvertes du lycée

Type de contenu :

Lien (URL)

Contenu du QR code :

www.exemple.fr/portes-ouvertes

Description (texte du PDF) :

Venez découvrir nos formations le samedi 14 mars de 9h à 17h. Scannez le QR code ci-dessous pour accéder au programme complet de la journée.

Taille du QR code (mm) :

60

Position du QR code :

Centre

Correction d'erreur :

Moyen (15 %)

☒ Afficher le contenu sous le QR code

Aperçu du QR code



Actualiser l'aperçu

0 %

Contenu encodé : https://www.exemple.fr/portes-ouvertes

Générer le PDF

Génération du QR code

- Le type de contenu valide et met en forme la saisie : `https://` ajouté aux liens, `mailto:` aux e-mails, `tel:` aux numéros.
- ZXing encode le texte en UTF-8 avec le niveau de correction choisi (L, M, Q, H).
- L'image est créée avec les couleurs du profil.

Génération du PDF

- Validation complète du projet.
- Génération du QR code en 800 px pour un rendu net.
- Création d'une page A4 avec la couleur de fond.
- Ajout du titre, de la description, du QR code (taille et alignement choisis) et du contenu encodé.
- Ajout des images à leur position absolue sur la première page.



4. Partie 1 – Fonctionnalités réalisées

Tâche du sujet	Réalisation
1. Interface Swing de saisie	Fenêtre à onglets : titre, type (texte, lien, e-mail, téléphone), contenu, description
2. Génération de PDF avec iText	Classe PdfGenerator
3. Génération du QR code	Classe QrCodeGenerator (ZXing), aperçu en direct
4. QR code intégré au PDF	Taille (20 à 180 mm), position (gauche, centre, droite), contenu affiché ou non
5. Gestion des erreurs	Exceptions personnalisées, messages dans des boîtes de dialogue (voir partie 6)
6. Tests unitaires	64 tests JUnit 5 (voir partie 7)
7. Documentation	Javadoc sur toutes les classes, README.md, ce rapport

5. Partie 2 – Évolution et nouvelles fonctionnalités

Tâche du sujet	Réalisation
Polices personnalisées	Polices standard (Helvetica, Times, Courier) ou fichier .ttf / .otf, intégré au PDF
Couleurs et styles	Couleur du titre, du texte, du fond de page, du QR code et de son fond. Gras, italique, souligné. Taille du titre et du texte
Ajout d'images	Bouton « Ajouter une image » (PNG, JPG, GIF, BMP), proportions conservées à l'ajout
Position et taille des images	Tableau modifiable : X, Y, largeur, hauteur en mm depuis le coin haut gauche
Sauvegarde des projets	Fichier .qrproj (JSON) : contenu, style et images
Sauvegarde des profils	Fichier .qrprofil (JSON) : police, tailles, styles, couleurs
Interface (facultatif)	Barre de progression, génération en arrière-plan (SwingWorker), aperçu en direct, raccourcis clavier, messages d'erreur détaillés
Tests et débogage	Tests ajoutés pour chaque nouvelle fonctionnalité
Documentation	Javadoc et rapport mis à jour

Changements dans le code entre les deux parties :

- Profil créé pour regrouper le style et pouvoir le sauvegarder seul.
- ImagePdf créé, Projet contient la liste des images.
- SauvegardeService ajouté.
- Vue découpée en trois onglets.

Fichier Profil Aide

Contenu Style Images

Nom du profil :

Profil par défaut

Police :

georgia.ttf (personnalisée)

▼

Police personnalisée (.ttf / .otf)...

Taille du titre :

24

Taille du texte :

12

Style :

☒ Gras
☐ Italique
☐ Souligné

Couleur du titre :

#C0392B

Couleur du texte :

#222222

Fond de la page :

#FFF8E1

Couleur du QR code :

#1A237E

Fond du QR code :

#FFFFFF

Charger un profil...

Sauvegarder le profil...

Aperçu du QR code

Actualiser l'aperçu

0 %

Contenu encodé : https://www.exemple.fr/portes-ouvertes

Générer le PDF

6. Gestion des erreurs

Toutes les erreurs héritent de `QrAppException`. Le contrôleur les attrape et affiche leur message.

Exception	Cas traités
<code>DonneesInvalidesException</code>	Contenu vide, URL, e-mail ou téléphone invalide, taille hors limites, couleur invalide, image introuvable ou qui dépasse de la page, police introuvable
<code>GenerationQrException</code>	Texte trop long pour un QR code, contraste insuffisant entre le QR code et son fond
<code>GenerationPdfException</code>	Dossier inexistant, fichier ouvert dans un autre programme, fichier qui n'est pas une police ou une image
<code>SauvegardeException</code>	Fichier introuvable, vide, corrompu ou avec des valeurs inconnues, écriture impossible

Autres protections :

- Le projet est validé avant de demander où enregistrer le PDF.
- Un PDF à moitié écrit est supprimé en cas d'erreur.
- Confirmation avant d'écraser un fichier existant.
- À l'ouverture d'un projet, les images ou la police déplacées sont signalées.

7. Tests unitaires

Lancement : clic droit sur `src/test/java` > **Run 'All Tests'** dans IntelliJ. Résultat : **64 tests, 0 échec**.

Classe de test	Nb	Ce qui est vérifié
<code>TypeContenuTest</code>	18	Mise en forme et refus des saisies invalides pour chaque type
<code>ProjetTest</code>	10	Validation du projet, du profil et des images
<code>CouleursTest</code>	3	Conversion hexadécimal ↔ couleur, luminance
<code>QrCodeGeneratorTest</code>	10	Taille, couleurs, contenu relu avec ZXing, texte trop long, contraste

Classe de test	Nb	Ce qui est vérifié
PdfGeneratorTest	12	PDF valide, texte présent, styles, police .ttf, 3 images dans la page, erreurs, progression
SauvegardeServiceTest	11	Sauvegarde puis rechargement identique, fichiers absents, vides ou corrompus

Le contenu du QR code est vérifié en relisant l'image générée avec le lecteur ZXing.

8. Défis rencontrés

Défi	Solution
Repère iText : origine en bas à gauche, en points	Saisie en mm depuis le haut à gauche, conversion dans PdfGenerator
QR code flou une fois agrandi dans le PDF	Génération en 800 px puis réduction à la taille voulue
QR code coloré illisible par un téléphone	Contrôle du contraste : modules plus foncés que le fond, écart de luminance minimum
Fichier de police invalide laissé verrouillé par iText (trouvé grâce aux tests)	Lecture de la police en mémoire avant de la donner à iText
Fenêtre figée pendant la génération	SwingWorker et barre de progression
Couleurs Java non sauvegardables en JSON	Couleurs stockées en code hexadécimal #RRGGBB
Anciens fichiers de projet sans certains champs	Valeurs par défaut au chargement